

SKILL WEEK7

TASK-1: To parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>

#define MAX_LINE 256

int main()
{
    int input_fd;
    int saved_stdin;
    char filename[100];
    char line[MAX_LINE];

    printf("=====\n");
    printf(" TASK-1: INPUT REDIRECTION\n");
    printf("=====\n");

    printf("Enter input file name: ");
    scanf("%99s", filename);

    /*
     * Open the input file
     */
    input_fd = open(filename, O_RDONLY);

    if (input_fd == -1)
    {
        perror("Error opening input file");
        printf("Missing or inaccessible file: %s\n", filename);
        return 1;
    }

    printf("\nFile opened successfully: %s\n", filename);
```

```

if (dup2(saved_stdin, STDIN_FILENO) == -1)
{
    perror("Error restoring stdin");
    close(saved_stdin);
    return 1;
}

close(saved_stdin);

printf("\n-----\n");
printf("Original stdin has been restored.\n");
printf("-----\n");

/*
 * Test that normal keyboard input works again
 */
printf("Enter a test message: ");

/*
 * Clear any leftover newline before reading
 */
int ch;
while ((ch = getchar()) != '\n' && ch != EOF)
    ;

if (fgets(line, sizeof(line), stdin) != NULL)
{
    printf("You entered: %s", line);
}

printf("\n=====\\n");
printf("TASK-1 completed successfully.\\n");
printf("=====\\n");

return 0;
}

```

```

laasya@laasya:~/skill_week7$ nano task1.c
laasya@laasya:~/skill_week7$ gcc task1.c -o task1
laasya@laasya:~/skill_week7$ ./task1
=====
TASK-1: INPUT REDIRECTION
=====
Enter input file name: input.txt

File opened successfully: input.txt

-----
Input has been redirected from the file.
Reading file contents through stdin...
-----

Welcome to Skill Week 7.
This is input redirection.
The program is reading this data from a file.
stdin has been redirected successfully.

-----
Original stdin has been restored.
-----
Enter a test message:
=====
TASK-1 completed successfully.
=====
laasya@laasya:~/skill_week7$ |

```

Task-2: To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>

#define MAX_LINE 256

int main()
{
    int output_fd;
    int saved_stdout;
    char filename[100];
    char message[MAX_LINE];

    printf("=====\n");
    printf(" TASK-2: OUTPUT REDIRECTION\n");
    printf("=====\n");

    printf("Enter output file name: ");
    scanf("%99s", filename);

    /*
     * Create/Open the output file
     *
     * O_WRONLY -> Open for writing
     * O_CREAT  -> Create file if it does not exist
     * O_TRUNC  -> Remove old contents
     */
    output_fd = open(filename,
                     O_WRONLY | O_CREAT | O_TRUNC,
                     0644);

    /*
     * Handle permission/file errors
     */

```

```

/*
 * Verify that output was written to the file
 */
printf("Enter a message to verify in the output file: ");

getchar(); // Remove newline left by scanf()

if (fgets(message, sizeof(message), stdin) != NULL)
{
    /*
     * Open the file again for verification
     */
    FILE *file = fopen(filename, "a");

    if (file == NULL)
    {
        perror("Error opening file for verification");
        return 1;
    }

    fprintf(file, "Verification message: %s", message);

    fclose(file);

    printf("Verification message written successfully.\n");
}

/*
 * Display final information
 */
printf("\nOutput file: %s\n", filename);
printf("File permissions used: 0644\n");
printf("Task-2 completed successfully.\n");

return 0;
}

```

```

=====
laasya@laasya:~/skill_week7$ nano task2.c
laasya@laasya:~/skill_week7$ gcc task2.c -o task2
laasya@laasya:~/skill_week7$ ls
input.txt  task1  task1.c  task1.co  task2  task2.c
laasya@laasya:~/skill_week7$ ./task2
=====
TASK-2: OUTPUT REDIRECTION
=====
Enter output file name: output.txt

Output file created/opened successfully: output.txt

=====
Original stdout has been restored.
=====
Enter a message to verify in the output file: Hello from Ubuntu
Verification message written successfully.

Output file: output.txt
File permissions used: 0644
Task-2 completed successfully.
laasya@laasya:~/skill_week7$ cat output.txt
=====
OUTPUT REDIRECTION TEST
=====
This message is redirected to the file.
stdout is currently connected to the output file.
The file was created using open().
The output was redirected using dup2().
Output redirection is working successfully.
Verification message: Hello from Ubuntu
laasya@laasya:~/skill_week7$

```

TASK-3: To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

#define MAX_FILENAME 100
#define MAX_MESSAGE 256

int main()
{
    int output_fd;
    int saved_stdout;
    char filename[MAX_FILENAME];
    char message[MAX_MESSAGE];

    printf("=====\n");
    printf(" TASK-3: APPEND OUTPUT REDIRECTION\n");
    printf("=====\n");

    /*
     * Parse the append operator >>
     */
    printf("Append operator: >>\n");

    /*
     * Get the output file name
     */
    printf("Enter output file name: ");
    scanf("%99s", filename);

    /*
     * Open the file in append mode.
     *
     * O_WRONLY -> Open for writing
     * O_CREAT  -> Create file if it does not exist
     * O_APPEND -> Add data at the end of the file

```

```

    * Validate append operation by reading the file.
    */
    printf("\n=====\\n");
    printf("VALIDATING APPEND OPERATION\\n");
    printf("=====\\n");

    FILE *verify_file = fopen(filename, "r");

    if (verify_file == NULL)
    {
        perror("Error opening file for validation");
        return 1;
    }

    printf("\\nCurrent contents of %s:\\n", filename);
    printf("-----\\n");

    char line[MAX_MESSAGE];

    while (fgets(line, sizeof(line), verify_file) != NULL)
    {
        printf("%s", line);
    }

    fclose(verify_file);

    printf("-----\\n");
    printf("Existing data has been preserved.\\n");
    printf("New data has been added at the end.\\n");

    printf("\\n=====\\n");
    printf("TASK-3 COMPLETED SUCCESSFULLY\\n");
    printf("=====\\n");

    return 0;
}
|

```

```
laasya@laasya: ~/skill_week7 x + v
Verification message: Hello from Ubuntu
laasya@laasya:~/skill_week7$ nano task3.c
laasya@laasya:~/skill_week7$ gcc task3.c -o task3
laasya@laasya:~/skill_week7$ ./task3
=====
TASK-3: APPEND OUTPUT REDIRECTION
=====
Append operator: >>
Enter output file name: append.txt

File opened successfully in APPEND mode.

-----
Original stdout has been restored.
-----
Enter a message to append: First message
Message appended successfully.

=====
VALIDATING APPEND OPERATION
=====

Current contents of append.txt:
-----

APPEND OPERATION
-----
This data is appended to the existing file.
Existing data is preserved.
The append operator is: >>
The file was opened using O_APPEND.
User message: First message
-----
Existing data has been preserved.
New data has been added at the end.

=====
TASK-3 COMPLETED SUCCESSFULLY
=====
laasya@laasya:~/skill_week7$ |
```

TASK-4: TO parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <errno.h>

#define MAX_FILENAME 100

int main()
{
    int error_fd;
    int saved_stderr;
    char filename[MAX_FILENAME];

    printf("=====\n");
    printf(" TASK-4: STDERR REDIRECTION\n");
    printf("=====\n");

    /*
     * Parse the error output filename
     */
    printf("Enter error output file name: ");
    scanf("%99s", filename);

    /*
     * Create/open the error file.
     *
     * O_WRONLY -> Open for writing
     * O_CREAT  -> Create if file does not exist
     * O_TRUNC  -> Clear old contents
     * 0644     -> File permissions
     */
    error_fd = open(filename,
                    O_WRONLY | O_CREAT | O_TRUNC,
                    0644);

```

```

/*
 * Verify the error file
 */
printf("\n=====\\n");
printf("VERIFYING ERROR FILE\\n");
printf("=====\\n");

FILE *verify_file = fopen(filename, "r");

if (verify_file == NULL)
{
    perror("Error verifying error file");
    return 1;
}

char line[256];

printf("\\nCaptured error output:\\n");
printf("-----\\n");

while (fgets(line, sizeof(line), verify_file) != NULL)
{
    printf("%s", line);
}

printf("-----\\n");

fclose(verify_file);

printf("\\n=====\\n");
printf("TASK-4 COMPLETED SUCCESSFULLY\\n");
printf("=====\\n");

return 0;
}

```

^G Help
 ^X Exit

^O Write Out
 ^R Read File

^W Where Is
 ^\\ Replace

^K Cut
 ^U Paste

^T Execute
 ^J Justify

```
laasya@laasya:~/skill_week7$ nano task4.c
laasya@laasya:~/skill_week7$ gcc task4.c -o task4
laasya@laasya:~/skill_week7$ ./task4
=====
TASK-4: STDERR REDIRECTION
=====
Enter error output file name: errors.txt

Error output file opened successfully.

-----
Original stderr has been restored.
-----
Error messages were redirected to: errors.txt

=====
VERIFYING ERROR FILE
=====

Captured error output:
-----
=====
ERROR STREAM TEST
=====
This is an error message.
stderr has been redirected to the file.
Failed to open file_that_does_not_exist.txt
Reason: No such file or directory
This error output is captured in the file.
-----

=====
TASK-4 COMPLETED SUCCESSFULLY
=====
laasya@laasya:~/skill_week7$ cat errors.txt
=====
ERROR STREAM TEST
=====
This is an error message.
stderr has been redirected to the file.
```

```
laasya@laasya:~/skill_week7$ nano task4.c
laasya@laasya:~/skill_week7$ gcc task4.c -o task4
laasya@laasya:~/skill_week7$ ./task4
=====
TASK-4: STDERR REDIRECTION
=====
Enter error output file name: errors.txt

Error output file opened successfully.

-----
Original stderr has been restored.
-----
Error messages were redirected to: errors.txt

=====
VERIFYING ERROR FILE
=====

Captured error output:
-----
=====
ERROR STREAM TEST
=====
This is an error message.
stderr has been redirected to the file.
Failed to open file_that_does_not_exist.txt
Reason: No such file or directory
This error output is captured in the file.
-----

=====
TASK-4 COMPLETED SUCCESSFULLY
=====
laasya@laasya:~/skill_week7$ cat errors.txt
=====
ERROR STREAM TEST
=====
This is an error message.
stderr has been redirected to the file.
```